

Detecting Cyber Threats with Machine Learning: A Comparative Analysis of Linear, Tree-Based, and Neural Models

Marti Geudens
r0934872
Piet Verguts
r0954535

Assistant-supervisor
Diwas Lamsal

Contents

Abstract	ii
List of Figures and Tables	iii
1 Introduction	1
2 Methods	3
2.1 Dataset and Features	3
2.2 Models	5
3 Results	9
3.1 Binary Classification Results	9
3.2 Multi-class Classification Results	13
4 Discussion	15
4.1 Interpretation of Findings	15
4.2 Contextualize with Literature	17
4.3 Limitations and Future Work	18
4.4 Updates based on the poster presentation feedbacks	18
5 Conclusion	19
5.1 Contributions	19
Bibliography	20

Abstract

This study evaluates the effectiveness of three distinct machine learning model families, Logistic Regression, Random Forest, and Neural Networks for network intrusion detection using the NSL-KDD benchmark dataset. Motivated by the increasing complexity of cyber threats and recent high-profile breaches, the research compares linear, ensemble-based, and non-linear approaches in both binary and multi-class classification scenarios. By employing a unified preprocessing pipeline and addressing severe class imbalance through class-weighted learning, the project evaluates model performance beyond overall accuracy, with particular focus on rare attack categories such as User-to-Root (U2R) and Remote-to-Local (R2L). Key results indicate that, while all models achieve high binary accuracy (up to 77.8%), their ability to generalize to unseen attack patterns in the test set varies significantly. Random Forest provides the best stability and ranking performance (ROC-AUC of 0.962), yet the Neural Network offers superior sensitivity to critical minority attacks, achieving a U2R recall of 0.70. However, the substantial performance drop between validation and test sets highlights that model expressiveness alone cannot overcome the inherent limitations of imbalanced training data. We conclude that, while Random Forest offers the most practical balance for general detection, effective multi-class intrusion detection depends on data-centric strategies, including hierarchical pipelines and anomaly-aware learning, to identify rare, high-risk security threats.

List of Figures and Tables

List of Figures

2.1	Class distribution training vs test	4
3.1	Normalized confusion matrix for binary Logistic Regression	10
3.2	Normalized confusion matrix for binary Random Forest	11
3.3	Normalized confusion matrix for binary Neural Network	12
3.4	ROC curves for binary intrusion detection on test set	12
3.5	Normalized confusion matrix for multi-class Logistic Regression	13
3.6	Normalized confusion matrix for multi-class Random Forest	14
3.7	Normalized confusion matrix for multi-class Neural Network	14

List of Tables

3.1	Binary intrusion detection performance on the test set.	9
3.2	Multi-class classification performance (General) on the test set.	13
3.3	Multi-class classification performance (Recall per class) on the test set.	13

Chapter 1

Introduction

In today's interconnected world, traditional rule-based Intrusion Detection Systems (IDS) struggle to keep pace with the rapid evolution of cyber threats. Attackers continuously adapt their methods by exploiting trust relationships, user behaviour, and system misconfigurations, rendering static detection techniques increasingly ineffective. Machine Learning (ML) has therefore been widely proposed as a suitable approach to address these challenges, and numerous benchmark studies demonstrate its effectiveness in this domain. For example, Abrar et al. show, using the well-known NSL-KDD dataset, that classifiers such as Random Forest (RF), Extra Tree, Decision Tree, and Support Vector Machines (SVM) can achieve very high detection accuracy for classifying normal versus malicious traffic, often exceeding 99% [1]. Furthermore, Gawand and Meesala confirm that NSL-KDD remains a useful benchmark for prototyping and comparative evaluation, despite the increasing complexity of real-world network traffic [2]. At the same time, NSL-KDD is a benchmark dataset and does not fully capture the characteristics of production networks. Hence, results obtained on this dataset should therefore be interpreted as comparative and methodological rather than indicative of real-world performance.

The need for adaptive intrusion detection is further reinforced by recent large-scale cyber incidents. In 2025, Ingram Micro suffered a ransomware attack by the SafePay group that resulted in the exfiltration of over 3.5 TB of data through compromised VPN credentials. Similarly, Orange SA was targeted by the Warlock group, while Louis Vuitton (LVMH) experienced a data breach via a third-party CRM vulnerability, exposing customer information across multiple European regions. These incidents share common characteristics, such as third-party security vulnerabilities, credential theft, and social-engineering-based intrusions, which frequently bypass traditional IDS solutions. This highlights the practical relevance of ML-based intrusion detection for both research and real-world deployment[3].

In this project, intrusion detection is formulated as a supervised classification task using the NSL-KDD dataset, in which preprocessed network connection records are provided as input to machine-learning models that predict whether network traffic is normal or malicious and, in extended experiments, identify the corresponding attack category (i.e. multi-class classification), including Denial of Service (DoS),

Probe, Remote-to-Local (R2L), and User-to-Root (U2R) attacks. This work builds upon prior research that has extensively evaluated ML techniques on NSL-KDD. Tavallaee et al. introduced NSL-KDD as a cleaned and rebalanced alternative to the original KDD Cup '99 dataset, addressing issues such as redundant records and biased attack distributions that previously led to inflated performance estimates [4]. Subsequent studies consistently report that tree-based ensemble models outperform many alternatives due to their ability to capture nonlinear feature interactions and handle class imbalance effectively [1, 5, 6]. Other authors have shown that, while deep learning models can achieve strong performance, classical machine-learning methods such as Random Forest often match or exceed their results while offering lower computational cost and greater interpretability [7].

Guided by these findings, this project evaluates multiple machine-learning models for intrusion detection on NSL-KDD, starting with Logistic Regression as a linear baseline, followed by a Random Forest classifier and finally a neural network to assess whether increased model expressiveness could yield additional benefits. By systematically comparing these approaches, the project aims to provide both empirical insight and practical guidance on the suitability of different ML techniques for network intrusion detection.

Chapter 2

Methods

2.1 Dataset and Features

This project uses the NSL-KDD dataset, a publicly available benchmark for intrusion detection research. The NSL-KDD dataset was introduced by Tavallae et al. [4] as a cleaned and improved version of the original KDD Cup 1999 dataset, addressing major shortcomings such as redundant records and biased attack distributions that previously led to overly optimistic detection results. The dataset is freely accessible through a public Kaggle repository [8] and is widely used for evaluating and comparing machine-learning-based intrusion detection systems [1]. The primary purpose of the NSL-KDD dataset is to support supervised classification of network traffic into normal and malicious behavior. Each data instance represents a single network connection and is described by a fixed set of features capturing various aspects of network activity. The classification task can be formulated either as binary classification (normal vs. attack) or as multi-class classification, where attacks are grouped into four main categories: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R), in addition to the Normal class.

The dataset consists of 125,973 samples in the training set (KDDTrain+) and 22,544 samples in the test set (KDDTest+), for a total of five classes. The class distribution is highly imbalanced, with DoS attacks dominating the dataset, while R2L and especially U2R attacks are extremely rare. This imbalance reflects realistic attack distributions but also introduces challenges for model training and evaluation, particularly in the multi-class setting [6]. Each network connection is described by 41 features, composed of a mix of numeric, categorical, and binary attributes. These include basic connection features (e.g., duration, protocol type, service, flag), content-based features (e.g., number of failed login attempts, file creation counts), and traffic-based features computed over time windows (e.g., connection counts, error rates, and bytes sent or received) [4]. The raw data is stored in text (.txt) format, with each row corresponding to a single connection record.

Figure 2.1 illustrates the class distribution of the NSL-KDD dataset for both the training and test splits. Severe class imbalance is evident in both partitions, with Normal and DoS traffic dominating the data, while R2L and U2R attacks occur

extremely rarely [4].

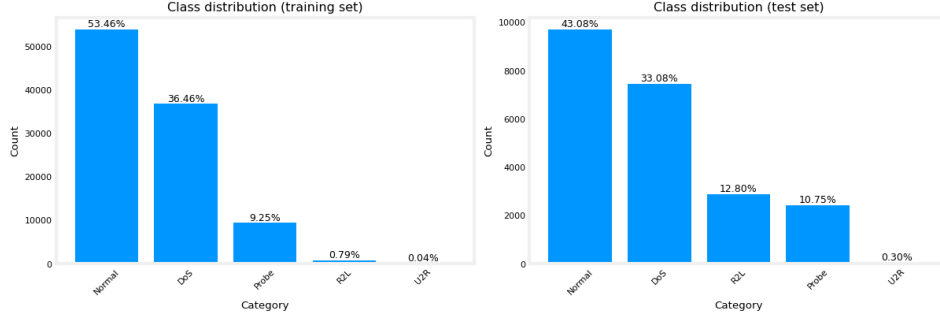


FIGURE 2.1: Class distribution training vs test

Importantly, the test set exhibits a different class composition than the training set. Although Normal and DoS remain the majority classes, the relative proportions of Probe and R2L attacks increase substantially in the test split. This shift reflects the intentional design of NSL-KDD, in which the test data includes attack types and distributions that are absent or under-represented during training in order to evaluate generalization to previously unseen intrusions [4]. This imbalance and distribution shift have two important implications. First, accuracy alone is a misleading evaluation metric, as strong performance on majority classes can mask poor detection of rare but security-critical attack types such as R2L and U2R. Second, models trained on the imbalanced training data are inherently biased toward dominant classes, making generalization to the test set more challenging. Consequently, the experimental evaluation focuses on macro-averaged metrics, minority-class (or attack-only) recall, and confusion-matrix analysis, which in turn motivates the use of class-weighted learning and more expressive, non-linear models.

Prior to model training, several preprocessing and feature engineering steps are applied. Categorical features, including `protocol_type`, `service`, and `flag`, are transformed using one-hot encoding, treating categories as nominal variables without imposing an artificial ordinal structure. Continuous features are standardized using z-score normalization (`StandardScaler`). To prevent data leakage, all preprocessing steps are fitted exclusively on the training subset and subsequently applied unchanged to the validation and test sets. Unseen categorical values in the validation or test data are handled by the encoder’s `ignore-unknown` implementation. The `difficulty_level` attribute introduced by Tavallaee et al. is excluded, as it represents meta-information rather than a network-traffic feature [4].

Furthermore, different model families may benefit from different feature representations. Tree-based models for example, can operate directly on ordinally encoded categorical variables, while linear and neural models typically require one-hot encodings. However, to ensure a fair and consistent comparison across the three models, a single unified preprocessing pipeline is used for all experiments. This design choice prioritizes consistency and comparability over model-specific optimization. As a result, performance differences can be attributed mainly to the modeling approaches

rather than to differences in feature representation. In practice, this unified dense representation primarily affects training efficiency for the Logistic Regression implementation due to the increased feature dimensionality introduced by one-hot encoding. Random Forest and Neural Network training proceed under the same feature representation without requiring separate preprocessing variants.

For model evaluation, the official `KDDTrain+` dataset is split into training and validation subsets, with the training subset used for parameter estimation and the validation subset used for model selection. The official `KDDTest+` dataset is reserved exclusively for final evaluation. By design, the test set contains attack types that are absent from the training data, providing a challenging benchmark for assessing generalization to previously unseen attacks [4]. Strict separation between training, validation, and test sets is applied for all models to obtain an unbiased estimate of model performance. Finally, while the preprocessing pipeline could be made fully reproducible by persisting fitted encoders and scalers using tools such as `joblib`, this project prioritizes conceptual clarity over production-level delivery. As a result, some best practices, such as persistent transformers, configuration files, and extensive exception handling, are not implemented. In a real-world deployment scenario, need for robustness, maintainability, and reproducibility would be essential components of the data-processing workflow.

2.2 Models

The project applies three machine-learning model families to the NSL-KDD intrusion detection task: Logistic Regression, Random Forest, and Neural Networks. These models include linear, ensemble-based, and non-linear approaches, enabling a structured comparison of interpretational capacity, robustness to (severe) class imbalance and generalization behavior, as often recommended in intrusion-detection benchmarking studies [1, 5, 6]. All models are trained and evaluated using consistent data splits and evaluation protocols to ensure comparability. Model performance is assessed using accuracy, macro-averaged F1-score, per-class recall, and confusion matrices. For binary classification tasks, ROC curves and AUC scores are additionally computed. Macro-averaged metrics are emphasized due to the strong class imbalance present in NSL-KDD [1, 4]. In addition to standard macro recall across all five classes, we also report attack-only macro recall (i.e. excluding Normal class) to better reflect performance across attack categories. This avoids the metric being dominated by the comparatively easy Normal class and provides a clearer view of attack-type sensitivity under class imbalance. No automated hyperparameter search (e.g. grid search or random search) was performed. Instead, model and training hyperparameters were selected through iterative, validation-driven refinement: candidate settings were chosen based on standard defaults and prior literature, then adjusted using validation-set performance and training diagnostics. This approach ensures methodological transparency while keeping the experimental scope aligned with the comparative nature of the project.

2.2.1 Logistic Regression

Logistic Regression serves as the baseline model and is directly adapted from the course lab assignments. Its primary objectives are to establish a transparent reference point, validate the preprocessing pipeline, and provide interpretable insights into feature relevance. Logistic Regression has frequently been used as a baseline for NSL-KDD due to its simplicity and interpretability [1, 5]. For binary classification, it models the probability of intrusion as follows, where σ denotes the sigmoid function. Model parameters are optimized by minimizing cross-entropy loss using gradient-based optimization.

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (2.1)$$

The original lab implementation was extended in two important ways. First, class-weighted training was introduced by augmenting the loss and gradient computations with a sample-weight term. This allows training errors on minority classes to be penalized more heavily, partially compensating for the severe imbalance between frequent attacks (e.g., DoS) and rare categories (e.g., R2L and U2R). This approach is widely used to mitigate class imbalance in intrusion detection, where rare attack categories would otherwise have negligible influence on the loss function [1, 6]. Second, multi-class support was added using a One-vs-Rest (OvR) strategy, a standard extension of binary classifiers for multi-class intrusion detection tasks. For a problem with K classes, OvR trains K independent binary classifiers, each distinguishing one class from all others. During inference, all classifiers are evaluated and the class associated with the highest logit value is selected. Stratified k-fold cross-validation was applied only for the binary Logistic Regression baseline to assess robustness under class imbalance. Logistic Regression was selected for this analysis due to its role as a baseline model. Random Forest and Neural Network models use a fixed train/validation split for model selection and the held-out test set for final evaluation.

Evaluation of the Logistic Regression model is conducted on a validation set using accuracy and macro-F1 as initial validation metrics, while class-wise recall and confusion-matrix analysis are emphasized in the final evaluation to account for class imbalance. For the binary formulation, ROC curves and AUC scores are computed from the predicted probabilities produced by the sigmoid output, as commonly reported in prior NSL-KDD studies [4, 7]. In the multi-class setting, normalized confusion matrices are used to analyze absolute error patterns. Furthermore, attack-only macro recall (i.e. excluding the Normal class) is reported to better reflect detection performance across attack categories.

2.2.2 Random Forest

Random Forest (RF) is chosen as the primary non-linear model because of its strong performance on NSL-KDD and its ability to capture complex feature interactions [1, 5, 6]. It handles mixed feature types and reduces overfitting through ensemble averaging. A Random Forest is an ensemble of decision trees trained on bootstrapped data samples, where each split uses a random subset of features. Final predictions are produced by majority voting across the trees.

The objectives of the Random Forest model are to improve multi-class detection performance relative to linear models, identify informative features, and build a more expressive classifier without excessive overfitting. RF is deliberately used in two distinct stages. In the first stage, an RF model is trained on the full 41-feature dataset to compute feature importance scores, a commonly used technique for feature ranking in intrusion detection. This model is used exclusively for feature ranking and not for final prediction. Based on these rankings, a reduced subset of the most informative features is selected. In the second stage, a new RF model is trained solely on this reduced feature set to serve as the predictive classifier. This two-stage approach enables an explicit evaluation of feature selection effects while maintaining a consistent learning algorithm. Class imbalance is addressed using class-weighted splitting criteria, as recommended in ensemble-based intrusion detection studies [1, 6]. Evaluation focuses on accuracy, macro-F1 score, and confusion matrices in both multi-class and binary formulations. A binary Random Forest model is additionally trained to isolate the fundamental intrusion-detection capability independent of category-level imbalance.

In our implementation, Random Forest models are trained with 300 trees and a fixed random seed (`random_state = 42`) to ensure reproducibility. Trees are allowed to grow without an explicit depth limit (`max_depth = None`), while training is parallelized across CPU cores (`n_jobs = -1`). Class imbalance is addressed through class-weighted learning: the multi-class feature selection Random Forest uses `class_weight = "balanced_subsample"`, which recomputes weights on each bootstrap sample, whereas the binary Random Forest uses `class_weight = "balanced"`, which derives weights from global class frequencies.

2.2.3 Neural Networks

Although neural networks are not always necessary or consistently superior to classical machine-learning techniques on intrusion-detection datasets, their ability to learn higher-order feature interactions makes them a valuable comparison point [7, 9]. Neural networks are therefore introduced as a third modeling family to assess whether their greater representational capacity provides advantages beyond linear and tree-based methods, as is often investigated in the intrusion-detection literature [1, 7, 9]. Fully connected feed-forward architectures are used, which are frequently applied to NSL-KDD and related datasets [1, 6, 7].

Several neural-network variants are evaluated to study the effects of regularization and imbalance-aware learning. The baseline multi-class model applies dropout regularization. Model 3MCv2 introduces L1 regularization, while Model 3MCv3 incorporates class weighting at the loss level to address class imbalance. An early stopping mechanism is applied based on validation loss. Binary neural-network variants mirror the multi-class configurations and are trained using binary cross-entropy loss. All Neural Network models use two hidden layers (128 and 64 units) with ReLU activations and dropout regularization (**Dropout** = 0.3). Multi-class models use a softmax output layer and are trained with sparse categorical cross-entropy using integer-encoded class labels. Training uses the Adam optimizer with mini-batches of 64 samples for up to 50 epochs. For binary neural-network models, decision thresholds are selected based on validation-set precision–recall characteristics, with the threshold maximizing macro-averaged F1-score used for final evaluation.

Chapter 3

Results

This section presents the experimental results obtained for the three evaluated model families: Logistic Regression (LR), Random Forest (RF), and Neural Networks (NN). Results are reported for both binary intrusion detection (Normal vs. Attack) and multi-class attack-category classification (Normal, DoS, Probe, R2L, U2R).

All models are trained using identical training, validation, and test splits, as described in Section 2.2. Validation performance is used exclusively for model selection and hyperparameter refinement, while test-set results are used for final evaluation and comparison. Performance is assessed using accuracy, macro-averaged precision, recall, and F1-score. For binary classification, ROC curves and AUC values are additionally reported. Macro-averaged metrics are emphasized due to the severe class imbalance present in the NSL-KDD dataset. In addition, attack-only macro recall is reported to more directly assess detection performance across attack categories. Minor numerical differences may occur across runs due to stochastic effects in training and evaluation. All metrics are therefore reported to three decimal places.

3.1 Binary Classification Results

Table 3.1 summarizes the binary intrusion detection performance of all three models on the test set. All metrics are macro-averaged unless stated otherwise.

TABLE 3.1: Binary intrusion detection performance on the test set.

Model	Accuracy	Macro Recall	Macro F_1	ROC-AUC
Logistic Regression	0.754	0.776	0.753	0.896
Random Forest	0.752	0.779	0.750	0.962
Neural Network	0.778	0.800	0.780	0.940

3.1.1 Logistic Regression

Logistic Regression is evaluated as a linear baseline for binary intrusion detection (normal vs. intrusive traffic). On the test set, the model achieves an accuracy of 0.754 and a macro-averaged F1-score of 0.753, with a ROC-AUC of 0.896. Class-specific recall is 0.933 for normal traffic and 0.619 for attack traffic, indicating strong detection of normal connections but reduced sensitivity to intrusive instances. Figure 3.1 shows the normalized confusion matrix for the binary Logistic Regression model on the test set.

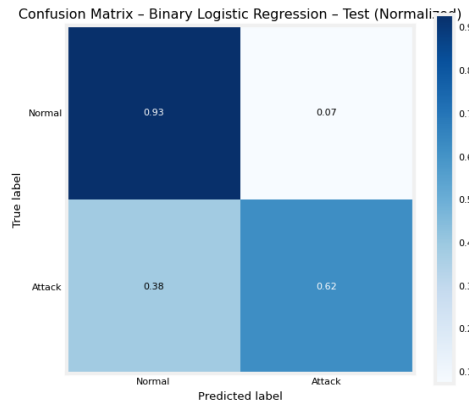


FIGURE 3.1: Normalized confusion matrix for binary Logistic Regression

In addition to the fixed train/validation/test evaluation, stratified 5-fold cross-validation is performed for the binary Logistic Regression baseline. Across folds, the model achieves a mean accuracy of 0.954 ± 0.001 and a mean macro-averaged F1-score of 0.950 ± 0.001 , indicating stable performance across different training subsets under class imbalance.

3.1.2 Random Forest

The Random Forest classifier is evaluated using a reduced feature representation consisting of the top-20 features selected via Random Forest feature-importance scores, as described in Section 2.2. This feature subset is used consistently across both binary and multi-class Random Forest experiments. On the binary test set, Random Forest achieves an accuracy of 0.752 and a macro-averaged F1-score of 0.750. Recall is 0.972 for normal traffic and 0.585 for attack traffic. The ROC-AUC on the test set is 0.962. Figure 3.2 presents the normalized confusion matrix for the binary Random Forest classifier on the test set.

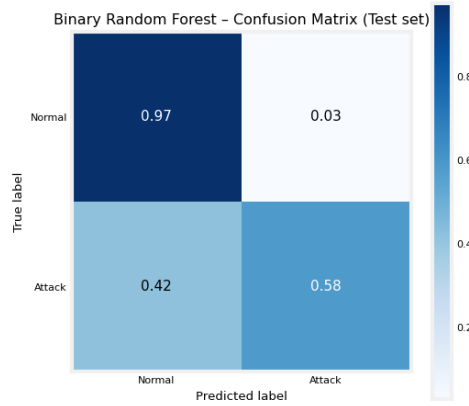


FIGURE 3.2: Normalized confusion matrix for binary Random Forest

3.1.3 Neural Network

Although neural networks are not always necessary or consistently superior to classical machine-learning techniques on intrusion-detection datasets, their ability to learn higher-order feature interactions makes them a valuable comparison point. Neural networks are therefore introduced as a third modeling family to assess whether their greater representational capacity provides advantages beyond linear and tree-based methods. On the binary test set, the Neural Network achieves an accuracy of 0.778 and a macro-averaged F1-score of 0.780, with a ROC-AUC of 0.940. Class-specific recall is 0.925 for normal traffic and 0.675 for attack traffic. Figure 3.3 shows the normalized confusion matrix for the binary Neural Network on the test set.

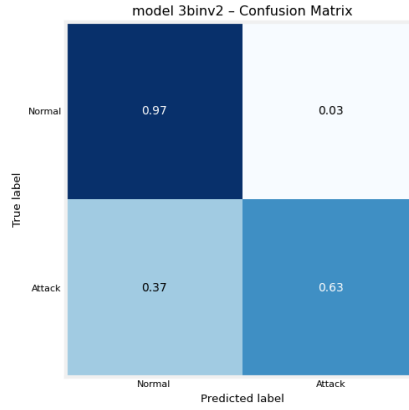


FIGURE 3.3: Normalized confusion matrix for binary Neural Network

Figure 3.4 compares the ROC curves of Logistic Regression, Random Forest, and Neural Network models on the binary test set, illustrating differences in discrimination performance across decision thresholds.

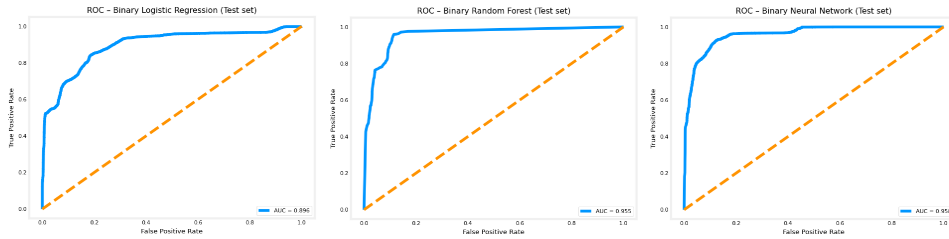


FIGURE 3.4: ROC curves for binary intrusion detection on test set

3.2 Multi-class Classification Results

Table 3.2 summarizes the multi-class classification performance of all models on the test set. Table 3.3 summarizes the recall per class on the testset.

TABLE 3.2: Multi-class classification performance (General) on the test set.

Model	Accuracy	Macro Recall	Macro F_1	Weighted F_1
Logistic Regression	0.753	0.687	0.678	0.742
Random Forest	0.736	0.472	0.474	0.688
Neural Network	0.792	0.681	0.580	0.770

TABLE 3.3: Multi-class classification performance (Recall per class) on the test set.

Model	Normal	DoS	Probe	R2L	U2R	Attack Recall
Logistic Regression	0.948	0.571	0.898	0.452	0.567	0.74
Random Forest	0.972	0.766	0.589	0.002	0.030	0.49
Neural Network	0.961	0.850	0.753	0.134	0.702	0.68

3.2.1 Logistic Regression

In the multi-class One-vs-Rest configuration, Logistic Regression achieves a test accuracy of 0.753 and a macro-averaged F1-score of 0.678. Performance varies substantially across attack categories, with higher recall for dominant classes (Normal, DoS, Probe) and reduced performance for the rare R2L and U2R classes. Figure 3.5 shows the normalized multi-class confusion matrix for the Logistic Regression model on the test set.

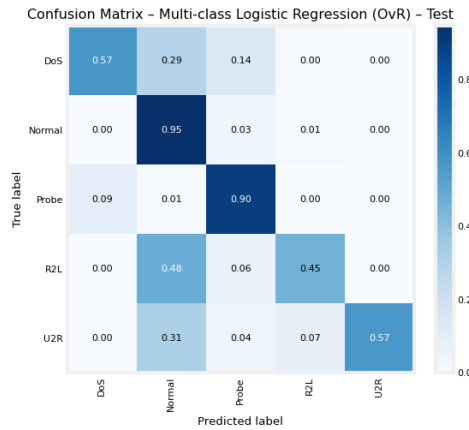


FIGURE 3.5: Normalized confusion matrix for multi-class Logistic Regression

3.2.2 Random Forest

In the multi-class setting, Random Forest is trained and evaluated using the same top-20 feature subset as in the binary task. On the test set, the classifier achieves an accuracy of 0.736, a macro-averaged F1-score of 0.474, and a weighted F1-score of 0.688. Figure 3.6 presents the normalized multi-class confusion matrix for the Random Forest classifier on the test set.

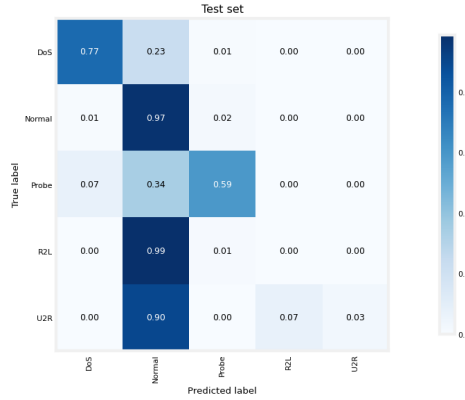


FIGURE 3.6: Normalized confusion matrix for multi-class Random Forest

3.2.3 Neural Network

For the multi-class classification task, the Neural Network is evaluated using the same experimental protocol as the other models. The reported configuration corresponds to the model selected based on validation performance and evaluated on the test set. On the test set, the model achieves an accuracy of 0.792, a macro-averaged recall of 0.681, and a macro-averaged F1-score of 0.580, with a weighted F1-score of 0.770. Figure 3.7 presents the normalized multi-class confusion matrix for the Neural Network on the test set.

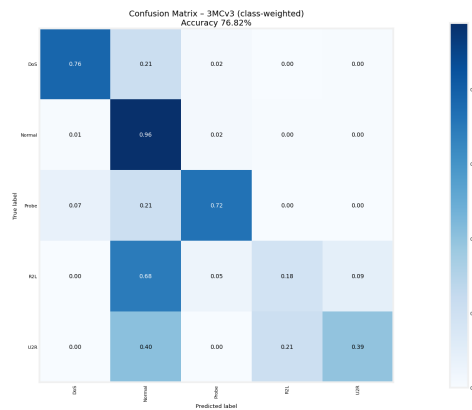


FIGURE 3.7: Normalized confusion matrix for multi-class Neural Network

Chapter 4

Discussion

4.1 Interpretation of Findings

This project compared Logistic Regression, Random Forest, and Neural Network models for intrusion detection on the NSL-KDD dataset using both binary and multi-class classification. While detailed performance metrics are reported only for the held-out test set, an important observation across all models is the substantial drop in performance from validation to test data, indicating limited generalization to unseen traffic patterns. Across all models, performance on the official NSL-KDD test set is notably lower than on the validation split, indicating limited generalization to unseen traffic patterns. This gap is expected, as the test set deliberately includes attack types and distributions that are under-represented or absent during training. Higher-capacity models, particularly Random Forest and Neural Networks, show a larger relative drop in performance, suggesting that strong validation results do not necessarily translate into robust test-set performance.

In the binary intrusion-detection task, all models achieve relatively high overall accuracy and favorable ROC characteristics on the test set. However, from a security perspective, false negatives are the most critical errors. They correspond to attacks that are incorrectly classified as normal traffic and remain undetected. Analysis of the confusion matrices and class-specific recall shows that Logistic Regression produces the highest number of false negatives among the evaluated models, reflecting a conservative decision boundary that prioritizes correct classification of normal traffic. Random Forest reduces the number of false negatives relative to Logistic Regression but still misclassifies a non-negligible fraction of attacks as normal, despite strong ROC-AUC values. The Neural Network achieves the lowest false-negative rate and the highest recall for attack traffic on the test set, indicating superior sensitivity to malicious behavior. These findings illustrate that ROC curves alone are insufficient for evaluating intrusion-detection performance, as they summarize ranking ability rather than actual operational decisions. Metrics that directly reflect missed attacks, such as recall for the attack class and the absolute number of false negatives, provide a more meaningful assessment of security effectiveness.

The generalization challenge becomes substantially more severe in the multi-class

setting, where the task shifts from distinguishing normal from malicious traffic to discriminating between highly imbalanced attack categories. This setting exposes significantly larger differences between models, particularly when evaluation emphasizes attack sensitivity rather than class frequency. When the Normal class is excluded from metric averaging, attack-only macro recall drops substantially relative to standard macro recall for all models, demonstrating that apparent performance on the multi-class task is largely driven by correct classification of dominant classes rather than reliable attack-type detection.

Logistic Regression achieves the highest attack-only macro recall among the evaluated models. However, this result reflects a tendency toward over-generalization rather than precise attack discrimination. While recall reaches approximately 0.45 for R2L and 0.57 for U2R, these gains are accompanied by very low precision, indicating that the model frequently assigns minority attack labels to non-rare samples. This behavior is consistent with the limitations of linear decision boundaries in a feature space where rare attacks overlap substantially with normal traffic, leading to increased false positives as the model attempts to compensate for class imbalance. In contrast, Random Forest exhibits the weakest performance on attack categories despite its strong results in the binary setting. Recall for R2L (≈ 0.002) and U2R (≈ 0.030) is near zero, indicating that these attacks are almost entirely misclassified as normal traffic. The confusion matrices confirm that Random Forest prioritizes stable separation of dominant patterns while effectively ignoring minority attack signatures. This outcome can be attributed to both extreme class imbalance and feature importance-based learning, which favors attributes for frequent classes and suppresses weak but security-critical signals associated with rare attacks.

The Neural Network provides the most balanced multi-class attack detection. Although recall for R2L remains modest, detection of U2R improves substantially (≈ 0.70), demonstrating that higher-capacity models can exploit subtle feature interactions that are inaccessible to linear or tree-based approaches. However, this improvement is accompanied by increased confusion among non-dominant classes, reflecting unstable decision boundaries in regions of the feature space with limited training data. This trade-off explains why macro-averaged F_1 -scores remain considerably lower than weighted metrics, despite improvements in minority recall.

Overall, these findings indicate that multi-class intrusion detection on NSL-KDD is constrained less by model choice than by data characteristics. Aggregate metrics obscure critical weaknesses in attack detection, while class-sensitive measures reveal that rare attacks remain difficult to identify. The results suggest that future improvements are more likely to come from data-centric strategies, such as hierarchical detection pipelines, selective resampling of minority attack classes, or anomaly-based modeling, rather than from increased model complexity alone.

In the binary neural network setting, the default decision threshold of 0.5 was not assumed a priori. Instead, decision thresholds were analyzed using validation-set precision–recall characteristics, and the threshold yielding the best macro-averaged F_1 -score on the validation set was selected for final evaluation. With the changed decision threshold, both models achieve a better accuracy for recalling attacks.

4.2 Contextualize with Literature

The results obtained in this study are consistent with established findings in the NSL-KDD literature, while also exposing limitations that are frequently masked by aggregate performance metrics. Prior benchmark studies consistently report very high overall accuracy for tree-based models. For instance, Abrar et al. demonstrate that Random Forest, Extra Trees, and Decision Tree classifiers achieve accuracies exceeding 99% on the NSL-KDD dataset when evaluated primarily using global accuracy and class-frequency weighted metrics [1]. Similar conclusions are reported by Hegde et al. and Hong et al., who identify Random Forest as the strongest performer across accuracy, precision, and ROC-AUC after preprocessing and feature selection [5, 6]. Our binary classification results are in line with these trends: all evaluated models achieve reasonably high accuracy and F_1 -scores, confirming that ML-based IDS can effectively separate normal from attack traffic under the NSL-KDD benchmark.

In contrast, when extending the analysis to the multi-class setting and emphasizing attack-centric metrics, our results diverge from much of the prior literature. While many earlier studies report macro-averaged metrics that include the dominant normal and DoS classes, our use of attack-only macro recall reveals substantially weaker performance on rare attack categories such as R2L and U2R. This outcome is consistent with the foundational analysis by Tavallaei et al., who highlight that NSL-KDD, despite improvements over KDD'99, remains highly imbalanced and that minority attack types are inherently difficult to model reliably [4]. In our experiments, Random Forest exhibits near-zero recall for R2L despite strong overall accuracy, whereas Logistic Regression attains higher recall for rare classes at the cost of severe over-generalization and very low precision. Neural Networks provide a more balanced compromise, improving minority-class recall while increasing confusion among non-dominant attack categories.

These differences can be attributed primarily to methodological choices. Many reference studies prioritize accuracy-driven model selection, employ aggressive feature selection, or rely on ensemble configurations that implicitly favor majority classes. In contrast, our evaluation explicitly isolates attack detection performance and assesses generalization to rare attacks using attack-focused metrics. As a result, this work complements existing research by demonstrating that high headline accuracy does not necessarily imply effective intrusion detection for infrequent but security-critical attack types. The key contribution of this project therefore lies not in exceeding reported accuracy levels, but in providing a more nuanced, attack-aware assessment of IDS performance. This finding is consistent with earlier critiques showing that aggregate accuracy can mask poor detection of rare and previously unseen attack types under class imbalance [4].

4.3 Limitations and Future Work

Despite the strong overall performance achieved by the evaluated models, several limitations limit the conclusions of this study. First, the NSL-KDD dataset remains highly imbalanced, with rare attack categories such as R2L and U2R representing only a small fraction of the data. This imbalance leads to substantial class overlap in the feature space and severely limits the ability of all evaluated models to detect minority attacks in the multi-class setting. Second, the fixed feature representation limits discrimination between rare attacks and normal traffic. Many security-critical signals are weak and are easily overshadowed by dominant patterns. Third, evaluation is limited to a single benchmark dataset. While NSL-KDD enables controlled comparison, it does not fully capture the diversity and dynamics of modern network traffic.

Future work should therefore prioritize data-focused strategies rather than increased model complexity. Feature selection techniques aimed at reducing class overlap, such as class-conditional importance ranking or embedded selection methods, could help isolate discriminative attributes for rare attacks. Hierarchical detection architectures are another promising direction. A staged pipeline could first separate normal from attack traffic, then classify broad attack categories, and finally identify specific subtypes. This approach can reduce the difficulty of direct multi-class learning under severe class imbalance. Finally, cost-sensitive or anomaly-aware learning methods could be applied. Weighted loss functions or hybrid anomaly-classification models can explicitly penalize missed U2R and R2L attacks and improve recall on these critical classes. These approaches directly address the main limitations of this study and would support more practical intrusion detection deployment.

4.4 Updates based on the poster presentation feedbacks

Following the feedback received during the poster presentation, several refinements were integrated into the study to enhance the quality of our analysis. A significant methodological update involved the additional use of one-hot encoding for categorical features, ensuring that the models could more effectively interpret non-numerical relationships within the data. To better illustrate the dataset challenges, an additional graph was included to compare class composition between the validation and test sets. It highlights the distribution shift between dominant traffic classes and minority attack types in the test data. The evaluation framework was also strengthened by prioritizing security-critical metrics. Specifically, the analysis now includes attack-only macro recall to better assess performance across imbalanced categories. This metric avoids the optimistic bias of standard averages. Furthermore, the results section was updated to place a deliberate focus on false negatives. By analyzing these undetected intrusions more closely, the study offers a more nuanced perspective on the operational risks and the sensitivity required for effective real-world intrusion detection.

Chapter 5

Conclusion

This study investigated the effectiveness of Logistic Regression, Random Forest, and Neural Network models for intrusion detection on the NSL-KDD dataset, addressing both binary and multi-class classification settings under severe class imbalance. The results show that, while all models achieve relatively high accuracy in the binary task, performance drops substantially on the official test set, highlighting limited generalization to unseen attack patterns. In the multi-class setting, aggregate metrics hide important weaknesses in rare attack detection, highlighting the need for attack-focused evaluation. Logistic Regression over-generalizes, Random Forest favors dominant patterns, and Neural Networks improve minority recall but introduce higher complexity and class confusion. Overall, Random Forest provides the most balanced trade-off between performance, interpretability, computational efficiency, and stability, despite its limited effectiveness on rare attack detection. The primary insight of this work is that model choice alone is insufficient to address multi-class intrusion detection challenges. Future progress will depend on improved data representations, class-aware evaluation, and validation under more realistic conditions.

5.1 Contributions

To meet the project deadlines, Piet and Marti coordinated their work through regular digital consultations. Due to diverging study and work schedules, as well as the physical distance between Antwerp and Leuven, online meetings proved to be the most efficient method for close collaboration. Contributions were shared across all project components. Depending on the topic, one team member initiated a section or experiment, after which the other iteratively reviewed, refined, and extended it until agreement was reached on the final quality. This structured and iterative approach ensured that all milestones were met while maintaining a consistent standard for both documentation and code quality.

Bibliography

- [1] I. Abrar, Z. Ayub, F. Masoodi, and A. Bamhdi, “A machine learning approach for intrusion detection system on nsl-kdd dataset,” in *Proceedings of the IEEE International Conference on Cyber Security (ICOSEC)*, pp. 919–924, 2020.
- [2] S. Gawand and D. Meesala, “Improved framework for detecting and predicting various cyber attacks using the nsl-kdd dataset,” *International Research Journal of Advanced Engineering Hub (IRJAEH)*, vol. 3, pp. 834–840, 2025.
- [3] PKWARE, “Data breaches 2025.” <https://www.pkware.com/blog/recent-data-breaches#:~:text=Ingram%20Micro,stolen%20or%20brute%20forced%20credentials>, 2025.
- [4] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the kdd cup 99 data set,” in *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications*, (Ottawa, ON, Canada), pp. 1–6, 2009.
- [5] R. Hegde, S. Likitha, and M. Natesh, “Intrusion detection system using machine learning based on nsl-kdd dataset,” in *Proceedings of the International Conference on Recent Advances in Science and Engineering Technology (ICRASET)*, (B. G. Nagara, Mandya, India), pp. 1–5, 2024.
- [6] R.-F. Hong, S.-C. Horng, and S.-S. Lin, “Machine learning in cyber security analytics using nsl-kdd dataset,” in *Proceedings of the International Conference on Technologies and Applications of Artificial Intelligence (TAAI)*, (Taichung, Taiwan), pp. 260–265, 2021.
- [7] M. L. Ali, K. Thakur, S. Schmeelk, J. Debello, and D. Dragos, “Deep learning vs. machine learning for intrusion detection in computer networks: A comparative study,” *Applied Sciences*, vol. 15, no. 4, p. 1903, 2025.
- [8] Kaggle, “Nsl-kdd dataset.” <https://www.kaggle.com/datasets/hassan06/nslkdd/data>, 2025.
- [9] S. Rawat, A. Srinivasan, and R. Vinayakumar, “Intrusion detection systems using classical machine learning techniques versus integrated unsupervised feature learning and deep neural network,” *arXiv preprint arXiv:1910.01114*, 2019.